

MINDPICK : 소비 패턴의 구조화로 실현하는 맞춤형 카드 추천 서비스

사용자 맞춤형 카드 추천 챗봇 구현 프로젝트

5조 유진 휘주 재혁 은혜

INDEX

목차보기

1

서론

2

데이터 수집 및 전처리

3

벡터 DB 생성 (칭킹 및 임베딩)

4

RAG 모델 구현 및 고도화

5

모델 복합 평가

6

최종 인사이트

관점의 차별화

사용자가 카드 추천 챗봇을 사용할 때
단순히 "어떤 카드가 좋아?"라는 질문에 대한 답변을 넘어
'서비스로서 지속적으로 머물게 만드는 가치'를 고민

1 서론: 서비스 상세 특징

01 전체 소비 데이터를 통합 분석하여
가장 범용적인 혜택을 주는 최종 카드 3종 추천



종합 분석

02 사용자의 소비 비중을 마인드맵 노드로 시각화하여
본인의 지출 우선순위를 직관적으로 인지시킴



구조화 시각화

03 마인드맵의 각 노드(식비/쇼핑 등)를 클릭 시,
해당 분야에만 압도적 혜택을 주는 특화 카드 추가 매칭



요소별 특화

2

데이터 수집 및 전처리

데이터 정형성 및 활용도를 극대화한 스키마 우선 설계 전략

기존 방식 (HTML 전수 수집)

과정 : 전체 HTML 수집 → 사후 파싱

효율 : 불필요한 데이터 포함, 전처리 복잡

유지보수 : 사이트 구조 변경 시 전체 로직 수정

우리의 방식 (스키마 우선 설계)

과정 : 최종 JSON 정의 → 필요 데이터만 매핑

효율 : 불필요한 로직 최소화, 즉시 활용 가능

유지보수 : 정의된 필드만 수정 → 유연

2

데이터 수집 및 전처리

데이터 저장 구조 : 용도에 따른 이원화 관리

CHROMA DB

메타데이터

내용 : 카드명, 혜택명, 혜택요약 등
용도 : 필터링 및 검색 정밀도

검색 결과와 함께 해당 정보의 출처를
제공할 수 있어 서비스의 신뢰도가 올라감

contents

내용 : 크롤링 한 모든 내용
용도 : 사용자 질의의 비교대상으로 사용

2

데이터 수집 및 전처리

단종 되지 않은 모든 카드 데이터 수집

주요 라이브러리 : SELENIUM

크롤링 한 카드

- 카드 고릴라에서 단종되지 않은 모든 데이터를 수집 (체크카드/신용카드 총 1184개)

동작 로직

- 전체 카드 리스트의 URL 수집 진행 후 URL 리스트로 각 카드의 상세 페이지 접속 후 데이터 수집
- 타켓 URL: 신용카드 800여개 + 체크카드 400여개

특이 사항

- 타겟 사이트가 VUE.JS 기반의 SPA페이지로 확인.
- 특정 요소가 나타날 때까지 기다리기 위해서 WEBDRIVERWAIT을 활용함

3

데이터 처리 및 벡터 DB 구축 (CHUNKING & EMBEDDING)

JSON → CONTENT → CHUNKING → EMBEDDING → CHROMADB

1. 수집된 JSON 데이터를 LLM이 이해하도록 LANGCHAIN DOCUMENT 형태로 재구성

정보 결합: 파편화 된 정보를 하나의 이야기로 통합

청킹 : RECURSIVECHARACTERTEXTSPLITTER 활용하여 데이터 분할

2. 데이터를 임베딩하여 CHROMADB에 보관

임베딩 : TEXT-EMBEDDING-3-SMALL 모델을 통해 데이터를 고차원 벡터로 변환

저장 구조 (CHROMADB)

4 RAG 모델 구현 및 고도화

우리 팀의 RAG 고도화 구조



4

RAG 모델 구현 및 고도화

프롬프트 고도화
COT+FEW-SHOT



AI에게 '**논리적 사고 절차**'와 '**답변 모범 예시**'를
함께 제공하여, 결과의 정확도와 일관성을 극대화

실제 구현된 코드

```
prompt = f"""
당신은 카드 추천 전문가입니다.
제공된 [카드 정보]를 바탕으로 아래 [사고 과정]에 따라 논리적으로 분석하여 답변하세요.

[사고 과정(Chain-of-Thought)]
1. 사용자의 주요 소비 패턴(영화, 카페, 쇼핑 등)과 요구사항을 파악한다.
2. 카드 데이터 중 해당 요구사항에 가장 부합하는 혜택을 가진 카드를 분석한다.
3. 선정된 카드들이 사용자에게 실질적으로 어떤 이득을 주는지 논리적 근거를 도출한다.
4. 최종적으로 서로 다른 카드 TOP 3를 선정하여 결과를 출력한다.

[답변 규칙]
- 반드시 서로 다른 카드 **TOP 3**를 순서대로 추천할 것.
- 문서에 없는 내용은 절대 지어내지 말 것.
- 답변은 아래 [예시]의 형식을 엄격히 따를 것.

[예시]
질문: 교통비 절약 카드 추천해줘.
사고 과정: 사용자가 광역버스 등 교통비를 최우선으로 고려함 -> A카드의 대중교통 20% 할인이 가장 적합함 -> 실질적 비용 절감 효과가 큼.
결과: 1. 추천 카드: A카드 / 주요 혜택: 대중교통 20% 할인 / 추천 사유: 매일 통근하는 사용자의 고정비를 가장 효과적으로 줄여줍니다.

[과거 대화 기록]
{formatted_history}

[카드 혜택 데이터]
{context}

[사용자 질문]
{query}
"""
```

CHAIN-OF-THOUGHT 구현 부분

[사고 과정(CHAIN-OF-THOUGHT)]

- 1. 사용자의 주요 소비 패턴(영화, 카페, 쇼핑 등)과 요구사항을 파악한다.
- 2. 카드 데이터 중 해당 요구사항에 가장 부합하는 혜택을 가진 카드를 분석한다.
- 3. 선정된 카드들이 사용자에게 실질적으로 어떤 이득을 주는지 논리적 근거를 도출한다.
- 4. 최종적으로 서로 다른 카드 TOP 3를 선정하여 결과를 출력한다.

FEW-SHOT 구현 부분

[예시]

질문: 교통비 절약 카드 추천해줘.

사고 과정: 사용자가 광역버스 등 교통비를 최우선으로 고려함

-> A카드의 대중교통 20% 할인이 가장 적합함

결과: 1. 추천 카드: A카드 / 주요 혜택: 대중교통 20% 할인 /

추천 사유: 매일 통근하는 사용자의 고정비를 가장 효과적

4

RAG 모델 구현 및 고도화

검색 고도화
COHERE RERANK



1차로 검색된 후보군 중 질문의 의도와
가장 부합하는 정답을 다시 한번 정교하게 **재정렬**

실제 구현된 코드

```
base_retriever = vectordb.as_retriever(search_kwargs={"k": 15})

my_rerank = CohereRerank(
    cohere_api_key=COHERE_API_KEY,
    model='rerank-v3.5',
    top_n=3
)

cohere_re_ranked_retriever = ContextualCompressionRetriever(
    base_compressor=my_rerank,
    base_retriever=base_retriever
)
```

광범위한 후보군 추출 (BASE_RETRIEVER)

코드: SEARCH_KWARGS={"K": 15}

사용자 질문과 유사도가 높은 데이터 15개를 벡터 DB에서 먼저 가져오기.

재정렬 (MY_RERANK)

코드: MODEL='RERANK-V3.5', TOP_N=3

1차로 뽑힌 15개의 후보를 COHERE RERANK라는 모델을 통해 의도를 파악해,
15개 중 가장 적합한 상위 3개를 다시 재정렬

압축된 리트리버 완성 (CONTEXTUALCOMPRESSIONRETRIEVER)

코드: BASE_COMPRESSOR=MY_RERANK, BASE_RETRIEVER=BASE_RETRIEVER

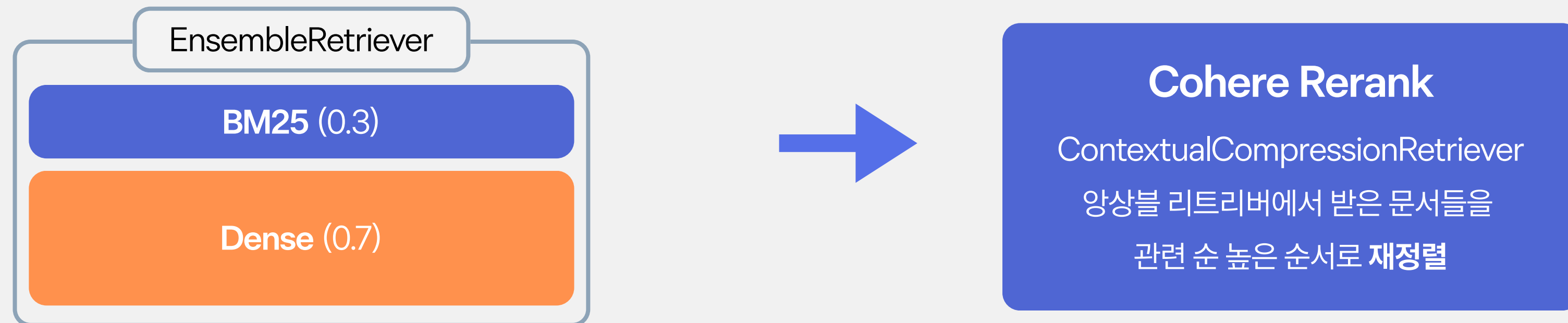
앞선 1차 검색기(RETRIEVER)와 2차 재정렬기(COMPRESSOR)를 하나로 묶어
최종적으로 '정제된 3개의 정보만 반환하는 리트리버'를 생성

4

RAG 모델 구현 및 고도화

BM25 하이브리드 전략

BM25 (Best Matching 25)는 정보 검색에서 가장 널리 사용되는 **키워드 기반** 검색 알고리즘.



- 카드사 혜택이 **대분류**로 분류된 데이터에서는 **키워드 기반 검색**이 효과적일 거라 판단하여 적용
- 단, BM25는 원천 데이터의 **특성상 한계**가 있음.
 - 사용자 질의에서 "카페" 키워드를 추출하더라도 "스타벅스", "커피빈" 등 브랜드명으로 직접 기술되는 경우가 많아 어휘 불일치 문제가 발생함.
 - 해외여행" 역시 "공항라운지", "진에어", "마일리지" 등으로 세분화되어 표현됨.
- 따라서 BM25 비중을 낮추되 상호 보완할 수 있게 **앙상블 리트리버로 묶음**: 코사인 유사도 검색 (0.7) + BM25Retriever (0.3)

모델 평가 단계

4

모델 평가 단계

Model based evaluation

평가 구성

- 두 모델이 동일한 페르소나 질의에 각각 응답을 생성한 뒤, 심판 모델(GPT-5-Mini)이 **1대1 대조 평가**
- 세트 구성 : 카드 3개를 추천하는 시스템으로 중복 응답이 많아 반복 횟수를 줄이고 페르소나를 늘림
 - 페르소나 **10명** : 10~70대 다양한 연령층으로 소비 패턴이 겹치지 않는 페르소나 생성
 - 페르소나별 질의·응답 횟수: **5회**
- 세트당 총 평가 횟수: 50회, 총 **3세트 진행**

평가 기준

- **추천 정확도** : 사용자 소비 패턴과 카드 혜택의 실제 일치도 평가
- **논리성** : 추천 이유가 사용자의 구체적 수치와 연결되는지 평가
- **관련성** : 추천 카드가 사용자 요구에 얼마나 부합하는지 평가
- **판정 결과**: 각 모델에 1~5점 부여 + 승자(또는 Tie) 선정
 - 점수 정의: 1~5 정수 (5=완전히 정확, 1=대부분 부정확/환각)



VS



> Model based evaluation : A vs C

Model-A :

- 프롬프트 고도화 : Few-Shot, CoT(금액 바인딩)
- 검색기 : BM25 + 코사인 유사도 + COHERE RERANK
- LLM : GPT-3.5-Turbo

Model-C :

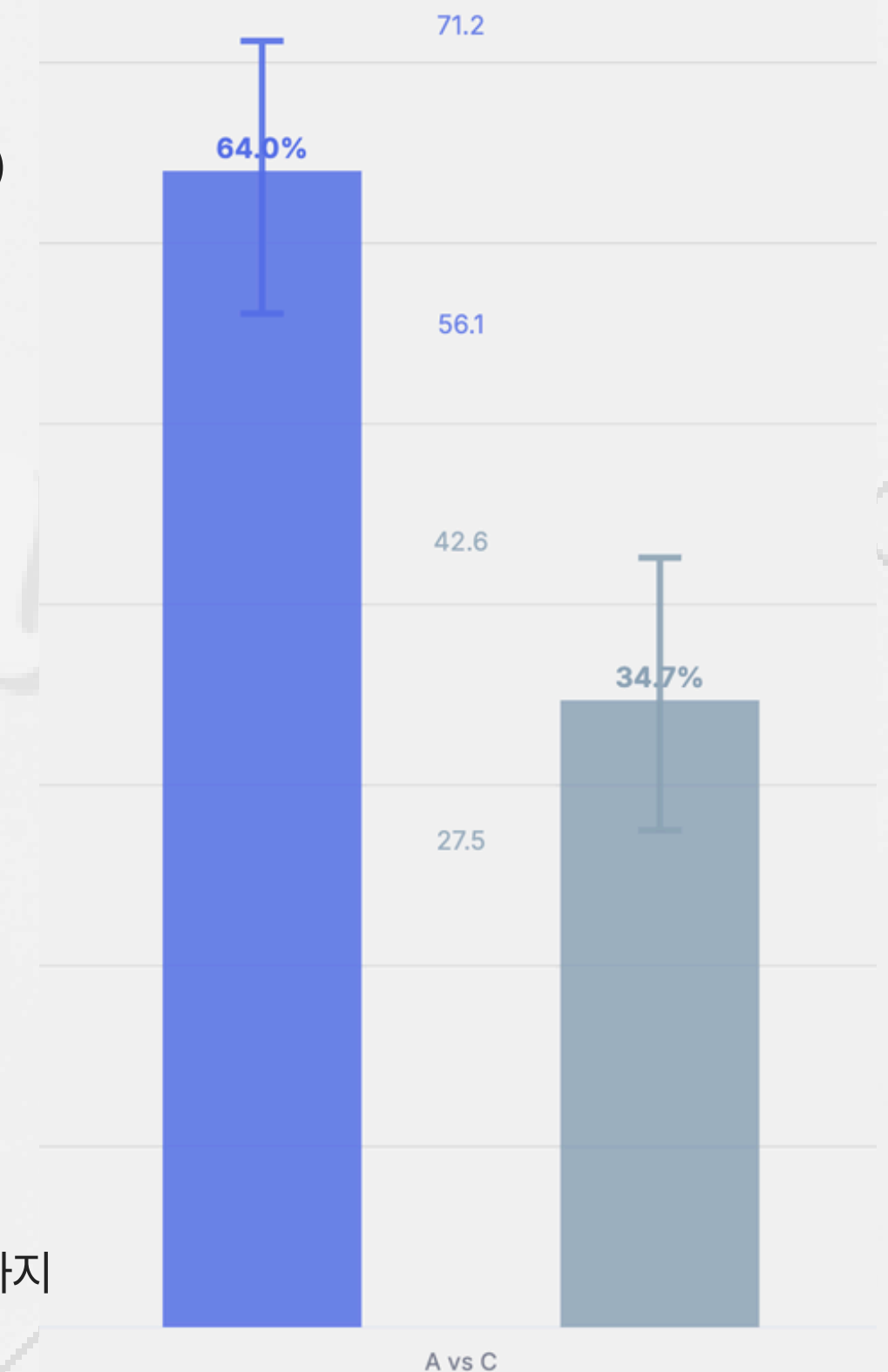
- 프롬프트 고도화 : Few-Shot, CoT(금액 바인딩)
- 검색기 : 기본 코사인 유사도 검사
- LLM : GPT-4o-Mini

결과

지표	Model-A	Model-C
승률	64	34.7
95% CI [Low]	57.4	26.9
95% CI [High]	72.5	41.9
평균 점수	3.41	2.98

결론

- GPT-3.5-turbo 기반 커스텀 모델이 GPT-4o-mini Pure 모델을 30.7%p 차이로 압도, 이는 **BM25, COHERE RERANK 등이 효과적으로 동작한다고 볼 수 있음**
- Pure 모델의 평균 점수 2.98, 업그레이드 LLM 모델이어도 원문 쿼리를 그대로 임베딩하면 주요 소비 패턴에 집중하지 못하고 **부가 혜택을 추천하는 경향**이 나타남.



4

모델 평가 단계

Code based evaluation

평가 구성

- 허깅 페이스의 오픈 소스로 공개된 임베딩 모델을 통해서 코사인 유사도 평가 진행
- 임베딩 모델 : bge-m3-korean
- **페르소나 선별** : 모든 페르소나의 답변을 작성하기엔 물리적 한계가 있어 3개의 페르소나로 압축함
- 평가 공정성을 위해 다음과 같이 페르소나 선별함
 - Pure 모델의 승률이 가장 높은 페르소나 : 10대 박열심 (승률 90%)
 - - 커스텀 모델의 승률이 가장 높은 페르소나 : 60대 김피트 (100%)
 - - 승률이 반반인 페르소나 : 50대 맞벌이 부부 김현우·이수진 (50% vs 46.7%, Tie 1)

평가 지표

- **유사도 점수** : bge-m3-korean 임베딩 → 코사인 유사도 (응답 vs 수기 정답지)
- **카드명 정확도** : 정답 카드 중 응답에 포함된 비율 (Recall)

> Code based evaluation : A vs C

결과

지표	Model-A	Model-C
평균 유사도	0.802	0.774
평균 재현율	0%	0%
유사도 기준 승률	86.70%	13.30%

결론

- 두 모델 다 **그럴듯한 텍스트(높은 유사도)**를 작성함, 하지만 **모두 재현율이 0%**로 그럴듯한 카드만 추천했다는 의미
- 코사인 유사도가 높더라도, 실제 카드 추천의 '정답'을 보장하긴 어려워 보임

실패할 수 밖에 없었던 평가?

- LLM식 기계적 서술과 인간의 **인지적 추론은 근본적으로 다름**
- 정답 작성 시 페르소나의 소비 패턴외에 지출 대비 월 최대 할인 한도, 적립 한도 등을 고려하여 추천 카드를 선별했으나 LLM은 이 부분을 놓쳐, 혜택을 과장되게 설명하거나 소비 비중이 높은 혜택을 강요?함
- 특정 모델의 예시 답안을 변형하여 작성하는 것이 효과적일 듯하나 주객전도가 되니 그냥 안하기로함

결과: 평가 폐기



> COHERE RERANK 좋네..

이상한 생각?

- BM25, COHERE RERANK 등 이미 잘 만들어진 **알고리즘, 컴포넌트를 사용하지 않고** 고도화하는 방법은 없을까 고민함
- BM25의 동작 방식 **키워드**에서 영감을 얻고 Metadata도 있으니 RDBMS와 같은 방식으로...?
 - ChromaDB 메타데이터('benefit_keywords')에 RDBMS식 조건 필터를 적용하여 관련 없는 카드가 유사도 상위에 올라오는 문제를 원천적으로 차단하기로 함

구현 과정

1. 소비 패턴 카테고리화 (최대 4개, percent 산출)
2. 카테고리명을 CATEGORY_MAP으로 **benefit_keywords** 확장
 - a. CATEGORY_MAP은 실제 카드고릴라의 혜택명의 집합
3. benefit_keywords로 교집합 우선 → 부족 시 합집합 풀백
 - a. \$contains, \$or로 필터링 문서 수집 → **교집합(set) 처리** → 문서가 목표 k보다 작은 경우 최초 \$or로 필터링으로 풀백
4. card_id pool 확정 (top 9) 후 Chunks 전체 수집
5. Cohere Rerank로 최종 선별 후 LLM 전달

> Base 모델 튜닝

오..쓸만한데?

- 로직을 구현하고나니 RDBMS같은 **Hard Filter + COHERE 조합**이 완성됨
- 테스트 했을 때, 나름 만족스러운 성능이 나옴 (Human Based Evaluation)

음...유사도 검색은 어디갔지?

- ChromaDB를 왜 쓰는건가라는 강한 의문이 남음, 그리고 좁혀진 **pool에서 COHERE가 효율적인가**라는 의문이 듦
- COHERE RERANK도 결국 유료서비스고 API 비용을 생각하여
- **COHERE를 버리고 좁혀진 Pool에서 유사도 검색(similarity_search)에 의존하기로함**

성능 평가

- 성능 평가 Model A(BM25+COHERE) vs Model B(HardFilter+유사도 검색) 진행
 - 승률 : 8: 2, 7:3으로 패배, 평균 점수 **2.78점**
 - **결과** : 처참하게 패배

> COHERE RERANK 좋네..3

패배 원인 분석

- LLM이 소비 패턴 **가중치와 교집합 1순위 키워드**에 심하게 **매몰**되는 현상이 생김
 - 교육비에 꽂히면 다른 소비 패턴은 무시하고 **교육 혜택만 카드만 나열함**
 - Judge(gpt-5-mini) 판정 사유 : Model B는 하나의 혜택만 추천해서 다양성이 떨어짐, 그러니 패배시킴 x 100 반복

패배를 인정한다.. 도와줘 CLADUE!!

- 3.5 Turbo를 정신차리게 하기위해 **CLAUDE**에게 조언을 구함 → **멀티 쿼리와 가중치 RRF 스코어링**을 써보라고 함
 - **멀티 쿼리** : 카테고리별 쿼리를 N개 생성하여 각각 검색함. 즉, 여러 관점에서 상위에 등장하는 카드를 선별할 수 있어 보였음
 - **RRF 스코어링** : 각 멀티 쿼리별로 가중치에 가변 Rank 스코어를 적용하여 가중치를 조절함
 - ``card_score[cid] += weight / (rank + 1)`
 - 비중이 높거나 검색 순위가 높을 수록 점수 배중
 - 복수 쿼리에서 중복 등장할수록 누적 점수 증가?

> Model based evaluation : A vs B

Model-A :

- 프롬프트 고도화 : Few-Shot, CoT(금액 바인딩)
- 검색기 : BM25 + 코사인 유사도 + COHERE RERANK
- LLM : GPT-3.5-Turbo

Model-B :

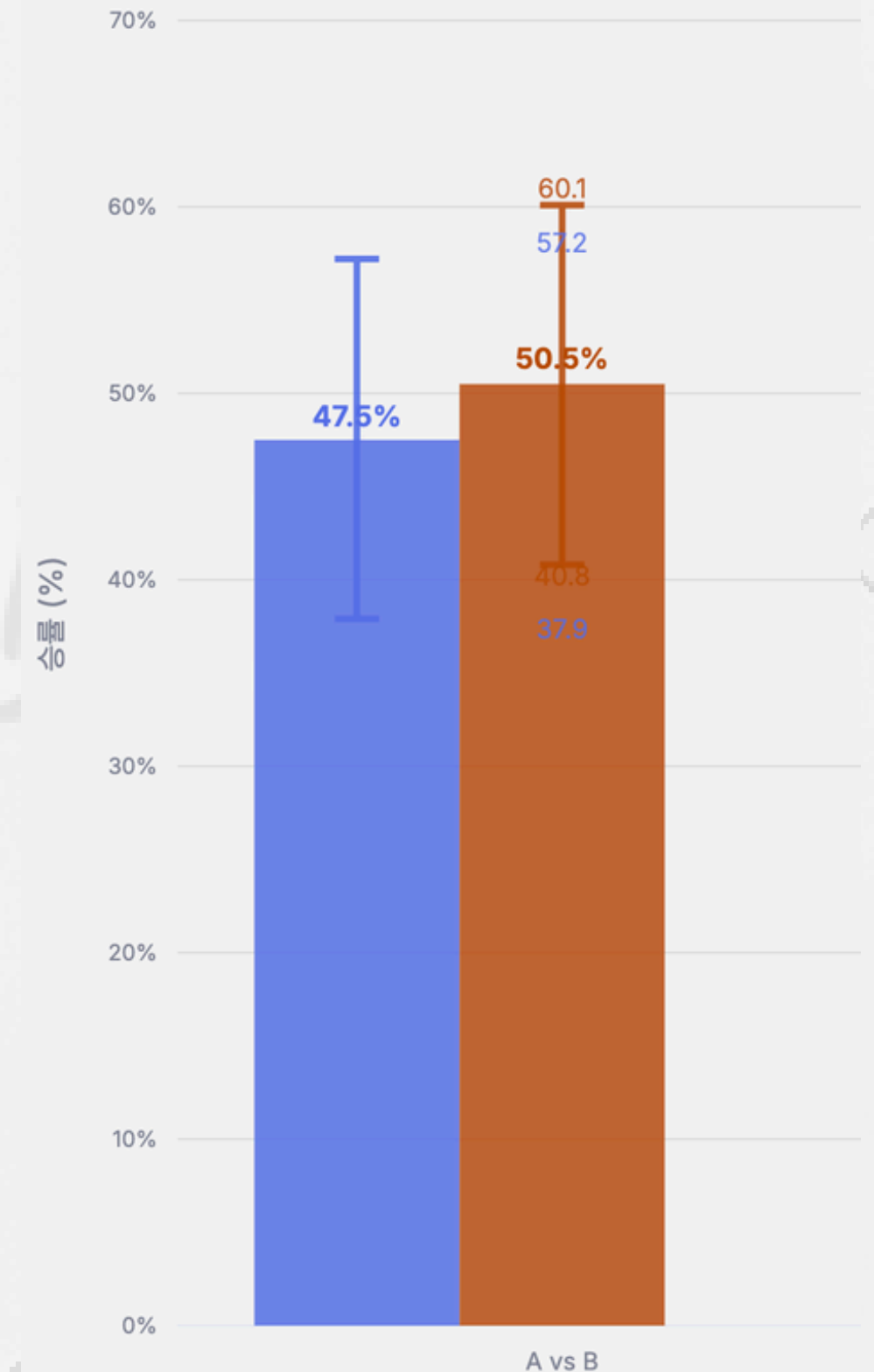
- 프롬프트 고도화 : Few-Shot, CoT(금액 바인딩)
- 멀티 쿼리
- 검색기 : HardFilter + 코사인 유사도 + RRF 가중치
- LLM : GPT-3.5-Turbo

결과

지표	Model-A	Model-B
승률	47.5	50.5
95% CI [Low]	37.9	40.8
95% CI [High]	57.2	60.1
평균 점수	3.15	3.11

결론

- A vs B 테스트는 100회 테스트 진행 : A모델과 B모델이 거의 비슷한 성능을 보여줌
- CI 95% 상당 부분 겹치는 부분이 있어, 어느 모델이 압도적으로 좋다고 보기 어려움
- 커스텀 모델(RRF 가중치 + 멀티쿼리)이 비슷한 성능을 낼 수 있다는 것에 만족
- 하지만 Claude에게 추천받은 RRF 스코어링은 작성자 본인의 **이해 부채로 남겨짐**, 그래도 성능이 좋으니 유지함



> Model based evaluation : B vs C

Model-B :

- 프롬프트 고도화 : Few-Shot, CoT(금액 바인딩)
- 멀티 쿼리
- 검색기 : HardFilter + 코사인 유사도 + RRF 가중치
- LLM : GPT-3.5-Turbo

Model-C :

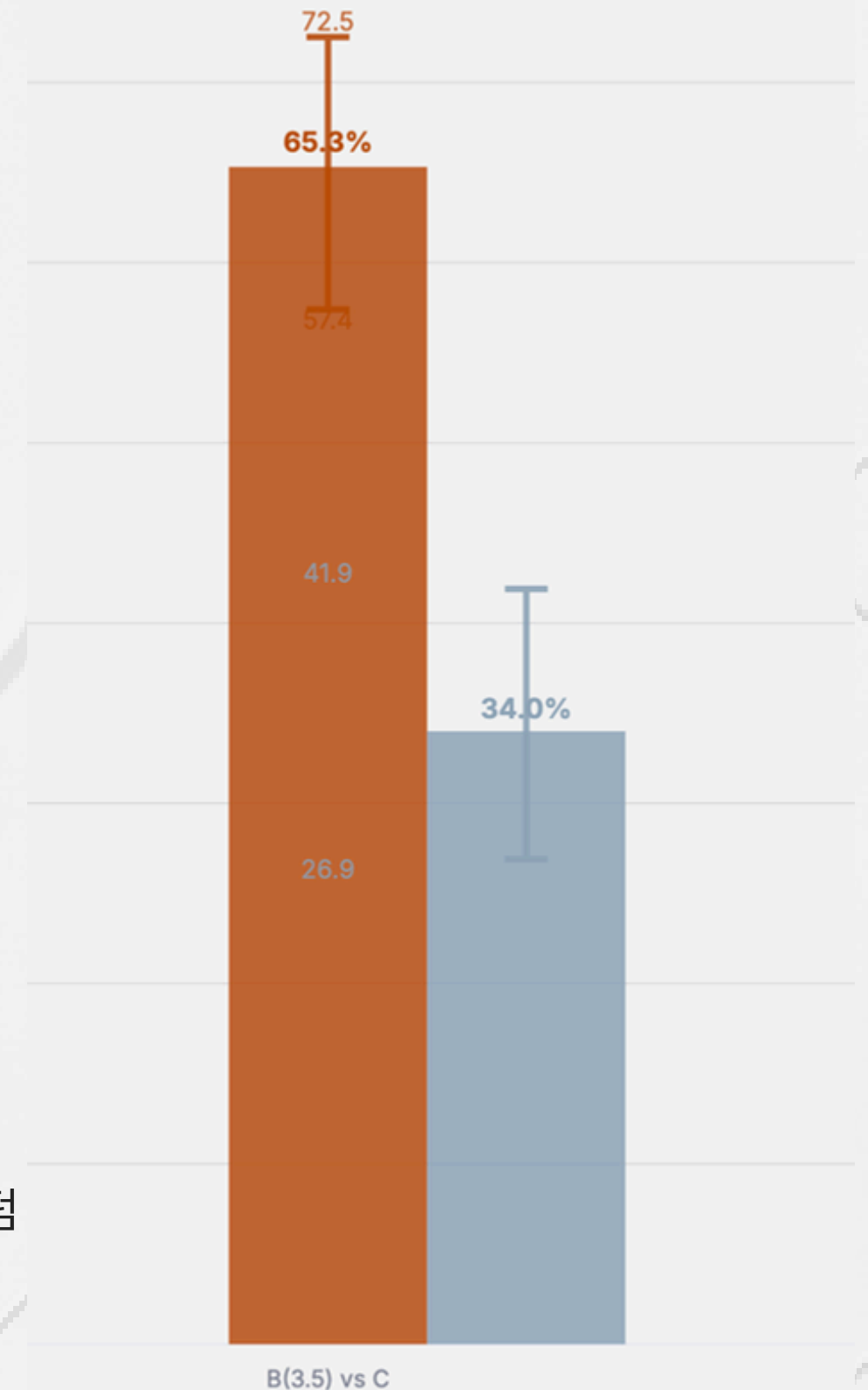
- 프롬프트 고도화 : Few-Shot, CoT(금액 바인딩)
- 검색기 : 기본 코사인 유사도 검사
- LLM : GPT-4o-Mini

결과

지표	Model-B	Model-C
승률	65.3%	34%
95% CI [Low]	57.4	26.9
95% CI [High]	72.5	41.9
평균 점수	3.38	2.92

결론

- B vs C 테스트는 A vs C 테스트와 동일하게 150회 조건으로 진행
- 95% CI: 3.5T = B 모델**의 하한 57.4%가 C모델의 상한 41.9%을 큰 폭으로 상회하고 CI 값이 전혀 겹치지 않아 커스텀 RAG 모델이 어떤 상황에서도 우위라는 점을 알 수 있음.
- COHERE RERANK, BM25 없이 커스텀 파이프라인 로직으로도 LLM 성능 격차를 상쇄하였다고 판단할 수 있음



> 번외 평가 테스트

Model-B에 4o mini를 얹으면 엄청난 성능이 나올 것 같은데??

- Base 모델(4o-mini)를 이기고 신남
- 커스텀 파이프라인 로직의 순수 성능을 알기 위해서 **LLM 모델을 동일하게 4o-mini**로 설정하고 테스트함

결과

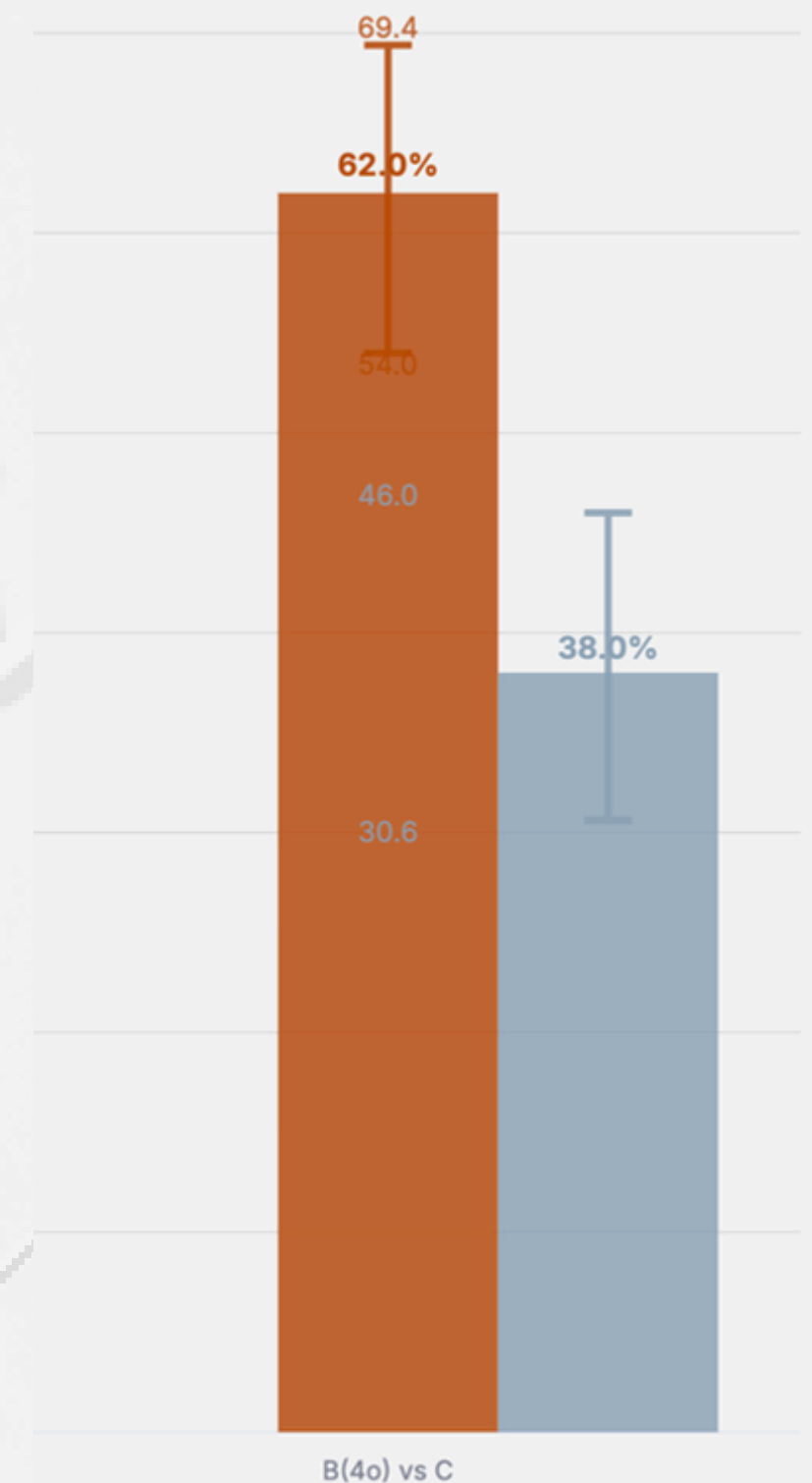
지표	Model-B(4o-Mini)	Model-C
승률	62%	38%
95% CI [Low]	54	30.6
95% CI [High]	69.4	46
평균 점수	3.42	3.01

결론 : 성능 차이가 전혀 없었다

- B(4o-mini) vs C 테스트도 150회 조건으로 진행
- HardFilter로 인해 좁혀진 Card Pool이 **LLM 성능을 제한**하는 것 같음, 추천 카드가 3.5Turbo와 별반 다름이 없음

GIGO와 같은 맥락

- 카드 추천 품질의 핵심 변수는 LLM 성능이 아니라 **검색 로직**과 **원천 데이터**의 품질을 좌지우지하는 것을 확인함
 - 증명: 같은 조건에서 GPT-5-Mini를 적용해도, 설명만 장황하게 할 뿐 추천 카드는 다르지 않았음



> 최종 인사이드

프로젝트가 남긴 것..

✓ 알고 쓰자

- » ChromaDB + LangChain의 코사인 유사도 검색은 수년간의 연구가 집약된 결과물
- » 고도화 기법을 덩지덩지 붙이기 전, Baseline의 튜닝을 최우선 과제로 적용이 필요했음
- » 고도화 기법도 신중히, 기법간의 시너지를 고려하여 적용 해야함

✓ GIGO

- » 검색 로직의 고도화 효과와 성능은 원천 데이터 품질에 따라 좌우됨
- » 모델의 고도화보다 더욱 중요한 건 데이터 수집 설계 전략

✓ RAG는 CHUNK다

- » 청킹 단위를 잘 설계하는 것이 검색 품질에 더 직접적인 영향을 줌
- » 즉, 청킹 전략이 임베딩 품질을 결정함

팀원들의 역할 분배 및 프로젝트 진행 흐름

유진

벡터 DB 생성 + RAG 모델 구현 + RAG 모델 고도화

휘주

웹크롤링 + 벡터 DB 생성 + RAG 모델 구현

은혜

STREAMLIT 구현 및 코드 조립

재혁

웹크롤링 + 평가 작업 + 모든 작업의 총괄

감사합니다